基于Scrapy框架的分布式网络爬虫系统设计

Design of distributed network crawler system based on Scrapy framework

何佳知
无锡工艺职业技术学院机电与信息工程学院，江苏　无锡　214200

摘要：为保证网络爬虫抓取效率和稳定性，通过应用 Scrapy 框架，完成对分布式网络爬虫系统的设计。首先，介绍网络爬虫工作原理、Scrapy 框架等相关技术。其次，在设计系统架构和系统数据库的基础上，分别设计代理池服务、实体管道、网页判重、网页下载等模块。最后，测试系统性能。结果显示，设计的系统具有网页抓取成功率高、运行效率高等特点，符合预期设计标准和要求，可以为相关人员提供有效的借鉴和参考。

关键词：Scrapy 框架；分布式；网络爬虫系统
中图分类号：TP393.09　　**文献标识码：**A

0　引言

在大数据时代背景下，各个行业产生了海量的数据，为保证万维网数据搜索性能，市面上出现多种多样的搜索引擎，搜索引擎在海量数据的快速搜集中发挥重要作用。同时，随着网络信息数据规模的不断增加和发展，用户对信息搜索引擎提出了更高的要求[1]。网络爬虫技术应运而生，该技术可以自动搜集和整理互联网信息数据，并向指定的数据库传输和存储相关信息[2]。但是传统单机网络爬虫存在抓取效率低、稳定性低、数据量不足等问题，无法满足大数据时代对海量数据的搜索需求。为解决以上问题，本文应用 Scrapy 框架，对分布式网络爬虫系统进行设计和应用。

1　系统设计关键技术

1.1　网络爬虫工作原理

网络爬虫作为一种重要的计算机程序，可以严格按照相关规则从互联网中抓取相关信息数据。网络爬虫工作原理如下：首先，借助有序的待爬行队列，存储各个统一资源定位符（uniform resource location，URL）队列。其次，严格按照相关队列顺序，从待爬行队列中抓取 URL，将网络请求发送至 URL 地址，从而获得所需要的网页内容，对该网页内容进行全面分析并提取新的 URL。最后，按照一定的顺序，将提取的 URL 存储至待爬行队列中，不断重复以上操作，当待爬行队列满足所设置好的爬行终止条件时，即可结束[3]。

作者简介：何佳知（1990—），女，硕士，副教授，研究方向为软件技术、云计算。

中国知网　https://www.cnki.net

1.2 Scrapy 框架

Scrapy 框架作为一种重要的应用框架，主要应用于结构化网络资源的提取和通用网络爬虫系统的构建。Scrapy 框架包含引擎、蜘蛛中间件、调度中间件、调度器、下载器中间件、下载器、项目管道等组成部分，通过应用异步网络库，可以保证网络通信处理质量和效率。目前，Scrapy 框架被广泛应用于数据挖掘、数据监测、系统测试等领域中，并取得了良好的应用效果 [4]。

2 系统总体设计

2.1 系统架构设计

如图 1 所示，系统架构被划分为以下 3 个部分。

（1）调度层。调度层可实时接收从 Scrapy 引擎中发送的请求，并将该请求直接存储至指定队列中，便于后期 Scrapy 引擎再次请求时可快速返回和获取。本文运用存储调度的方式，对远程字典服务（remote dictionary server，Redis）访问请求进行调度，从而达到智能化调度和爬取分布式任务的目的。本文系统借助调度层，可以实现对单个 URL 链接的提取和封装，当 URL 封装成指定的请求时，需将该请求直接发送至下载器，由下载器搜索和下载所需要的资源。在进行爬虫解析请求时，可解析出相应的实体，并将该实体发送至实体管道，由实体管道对该实体采取进一步处理 [5]。当 URL 链接成功解析后，可向调度层发送解析后的 URL 地址，由调度层等待抓取。

（2）中间件层。中间件层主要包含 Cookies 中间件、Proxy 中间件、UA 中间件、Retry 中间件等组成部分。其中，Cookies 中间件主要用于对用户本地终端数据的安全化存储，方便后期系统快速辨别不同用户的身份信息，保证会话跟踪实现效果；Proxy 中间件主要用于对代理池进行对接处理，方便系统快速识别不同目标网站服务器的 IP 地址；用户代理（user agent，UA）中间件主要用于对添加错误 UA 信息的实时修改和更新；Retry 中间件主要用于对运行异常代理的重写和改进。

（3）数据层。数据层主要利用 MongoDB 数据库，获取、清洗和融合处理数据。整个数据层在具体设计时，主要继承 Spider 类，该 Spider 类定义了如何爬取某个网站，应用该子类可以对请求方法进行重写，并实时选择和抓取数据模式和日期。

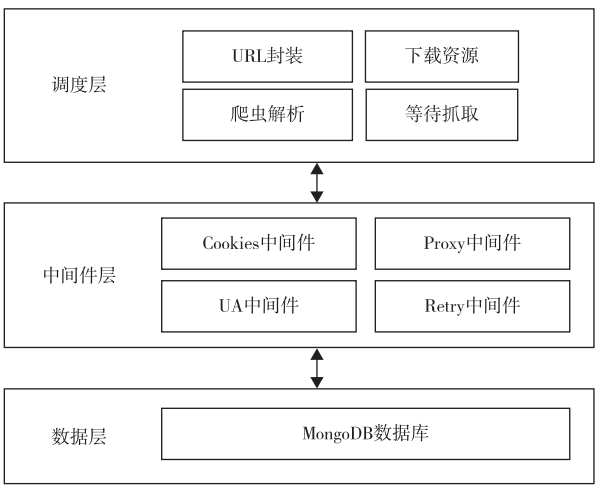

图1 系统架构设计

2.2 系统数据库设计

在本文系统中，主要运用 MongoDB 数据库，从而实现长时间存储数据。在设计 MongoDB 数据库时，需设计合理的数据表结构，保证系统数据库的稳定性和可靠性。在网络爬虫期间，主要目标网站为微博和脉脉。对于微博目标网站，重点关注微博条目详细信息、用户详细信息等数据；对于脉脉目标网站，重点关注匿名消息条目详情、反馈消息等数据。因此，本文系统设计了微博条目表、微博用户表、脉脉详细表。其中，微博条目表主要包含微博编号、点赞量、转发量、文本、原始文本、创建实际等属性；微博用户表主要包含微博编号、用户名、头像路径、用户描述、粉丝数量等属性；脉脉详细表主要包含数据源、评论量、插入时间、消息类型、发布时间、站点名等属性。

3 系统功能模块设计

本文系统主要用于对微博、脉脉目标网站相关数据进行实时爬取。整个系统功能模块经过细化

中国知网 https://www.cnki.net

后，得到代理池服务、实体管道、网页判重、网页下载等模块，各个模块设计流程如下。

3.1 代理池服务模块设计

代理池服务模块主要包含以下 4 个子模块：①获取模块。在获取模块中，用户可以借助本文系统提供的代理网站，抓取需要的代理信息。代理既可以免费使用，也可以付费使用。整个获取模块中，可以从多个数据来源中获取所需要的数据，当数据被成功抓取后，可以将其直接传输和存储至数据库。②存储模块。存储模块主要用于代理信息的统一存储和抓取，为保证代理的唯一性，需标识不同代理的处理状态。③检测模块。在检测模块中，用户可以借助存储模块，对系统数据库中的代理信息进行统一检测，结合最终检测结果，标识和设置不同代理相应的分数状态。④接口模块。接口模块可以直接访问系统数据库，快速获取需要的数据，但是这种操作缺乏安全性，容易导致数据库内配置信息丢失、泄露等。为解决以上问题，本文重点设计接口模块，用户借助接口模块，可以安全、快速地获取需要的代理信息，保证代理信息获取机会的均等性，提高本文系统整体负载均衡能力。

3.2 实体管道模块

实体管道模块主要用于对爬取数据的清洗、验证和存储等相关处理。该模块主要包含微博实体管道、脉脉实体管道等多个子模块，各个子模块之间既相互独立，又相互依赖，按照流水线方式进行有序运行。其中，微博实体管道模块主要用于实时添加爬取时间字段，当该时间字段信息添加完成后，本文系统可以自动将添加好的时间字段信息存储至 MongoDB 数据库；脉脉实体管道模块设计目的是保证脉脉数据的独特性和唯一性，避免脉脉数据重复，当脉脉数据经过去重、清洗处理后，可以将该数据直接存储至 MongoDB 数据库，便于其他人员查看和调用。

3.3 网页判重模块

在网络爬虫运行期间，为避免因同一页面被多次下载而增加系统运行时间和运行负荷的情况，本文重点设计和实现网页判重模块，确保 URL 下载队列含有唯一的 URL 和爬取 URL 的唯一性。URL 网页判重工作具体流程为：首先，利用本文系统完成对单个全局变量的构建，检测和判断某一 URL 是否被浏览和访问过；其次，还要对当前待爬取 URL 进行探测，探测其是否处于全局变量中。从图 2 可以看出，运用哈希算法，获取所需要的 URL 地址，并且对信息摘要算法（message-digest algorithm）MD5 进行压缩映射处理，保证 URL 去重池功能的实现效果。在具体网络爬虫期间，运用 MD5 算法，可以对任意位数的字符串进行压缩，使其成为 128 位整数，并将该整数统一映射为指定的物理地址，确保 URL 去重池具有一定的独特性和唯一性。在单次爬取期间，如果映射物理地址出现重复，需要检验 MD5 在存储期间是否出现碰撞，如果出现碰撞，需对 URL 中的二进制数（10000）进行左移一位，得到 100000；如果未出现碰撞，需丢弃 URL 地址。

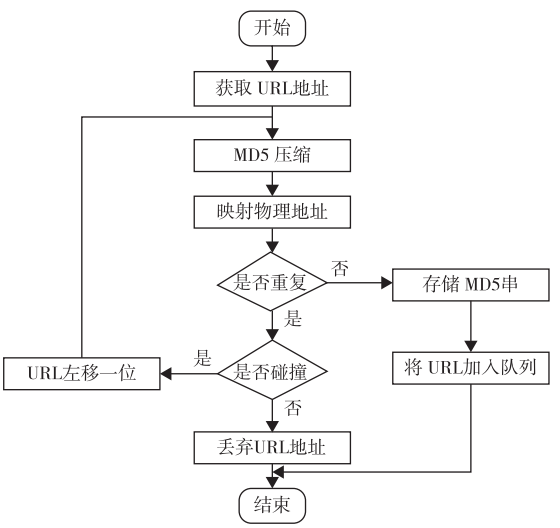

图2 网页判重流程

3.4 网页下载模块

在具体设计网页下载模块时，技术人员需要应用 Scrapy 框架内部的 Downloader Middleware 类，该类定义了一个或多个方法的类，运用该类可以对 process_request 方法进行重写。本文系统从指定的请求队列中，完成对特定 URL 地址的提取，并将

中国知网 https://www.cnki.net

其发送至下载器中间件，由下载器中间件对所需要的网页内容进行下载，如果网页内容下载期间出现错误问题，需调用Rotate User Agent Middleware类，对网页内容进行重新下载，整个下载过程重复3次，当重复下载 3 次结束后，仍然下载失败，则需借助失败队列存储 URL 地址和相关字符。网页下载流程如图 3 所示。

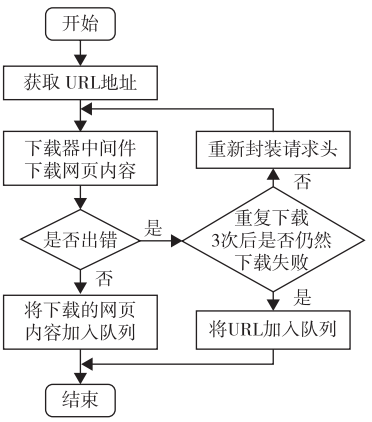

图3　网页下载流程

4　系统测试

为验证本文系统的高效性和稳定性，本文以“hao123”为测试页面，并将本文系统和传统单机网络爬虫系统分别设置为实验组和对照组。在实验组中，主要用到 url.py 文件，该文件在具体使用中，通常产生大量的 URL 队列，此时，需利用 Redis 数据库完成对以上 URL 队列的存储；同时，应用 Scrapy crawl 命令，保证分布式抓取执行效率和效果；最后应用 MongoDB 数据库，存储相关数据。在对照组中，主要运用传统 Nutch 网络爬虫，抓取和存储相关数据。在进行网页数据抓取期间，本文分别测试实验组、对照组成功下载保存网页数量，网页抓取统计结果如表 1 所示。从表 1 中的数据可以看出，与对照组相比，实验组在进行网页抓取时，网页抓取成功率和运行效率明显得到提升，访问时间有效缩短，这说明本文系统在网页抓取和运行方面具有显著优势。

表1　网页抓取统计结果

测试项		测试次数			
		第1次	第2次	第3次	第4次
理论抓取页数/个		2 156	3 568	6 079	7 820
实验组	抓取速率/（个·s^{-1}）	2 086	3 489	5 987	7 762
	用时/s	205	310	420	564
对照组	抓取速率/（个·s^{-1}）	1 820	3 058	5 744	7 421
	用时/s	850	1 598	2 531	3 429

5　结语

综上，本文应用 Scrapy 框架所设计的分布式网络爬虫系统在改进和优化传统 Scrapy 框架的基础上，对 Scrapy 框架分布式能力进行不断拓展和提升，不仅保证网络爬虫分布式抓取效果，还能实现对相关数据的非结构化存储。本文系统具有网页抓取成功率高、运行效率高等特点，表现出一定的应用价值和应用前景，值得被进一步推广和应用。

参考文献

[1] 刘多林，吕苗 . Scrapy 框架下分布式网络爬虫数据采集算法仿真 [J]. 计算机仿真，2023，40（6）：504-508.
[2] 卢照，师军，张耀午，等 . 基于双缓冲的分布式爬虫调度策略的设计与研究 [J]. 计算机与数字工程，2022，50（8）：1686-1690.
[3] 何西远，张岳，张秉文 . 基于分布式爬虫的微博舆情监督与情感分析系统设计 [J]. 现代信息科技，2024，8（5）：111-114，119.
[4] 李代祎，谢丽艳，钱慎一，等 . 基于 Scrapy 的分布式爬虫系统的设计与实现 [J]. 湖北民族学院学报（自然科学版），2017，35（3）：317-322.
[5] 方奇洲，程友清 . 基于 Docker 容器的分布式爬虫的设计与实现 [J]. 电子设计工程，2020，28（8）：61-65.

中国知网　https://www.cnki.net